CSE 816 · SOFTWARE PRODUCTION ENGINEERING · MINI PROJECT

Calculator REST API DevOps Pipeline

A Java and Spring Boot calculator packaged as a container and deployed with Docker Compose, with Jenkins pipeline configuration for automated build and delivery.

Course: CSE 816 — Software Production Engineering

Stack: Java 21 · Spring Boot · Maven · Jenkins · Docker · Docker Compose · Nginx

Project: SPE Mini Project — End-to-End CI/CD Pipeline for a Calculator REST API

Report date: 30 October 2026

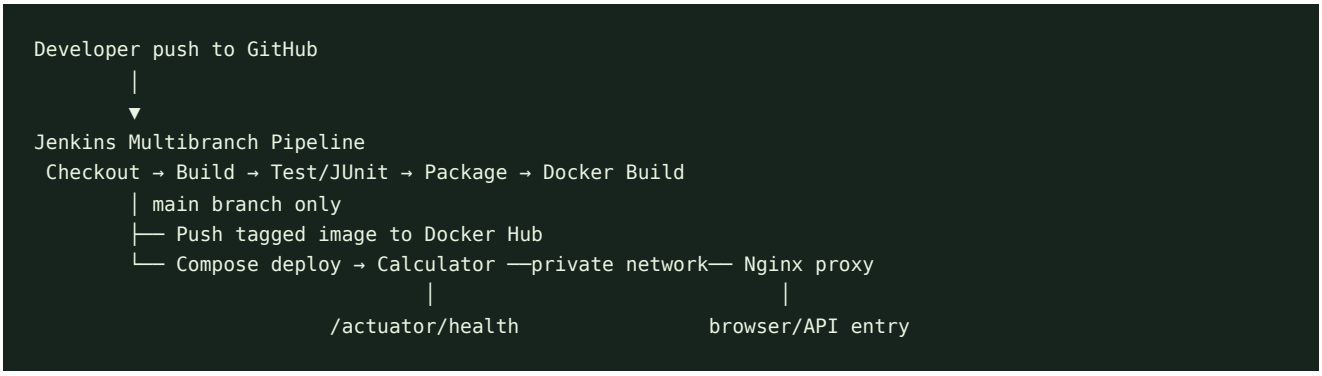
Project summary

The service provides addition, subtraction, multiplication, and division through REST endpoints. It accepts decimal operands, returns a JSON result, rejects missing or invalid parameters with structured HTTP 400 errors, and exposes Spring Actuator health status.

The deployment uses two Compose services. The calculator runs on the private Compose network; Nginx provides the browser and API entry point. Both services have health checks and restart policies. The application image uses a Maven build stage and a smaller Java runtime stage, runs as a non-root user, and exposes port 8081 inside the container.

> **Observed local deployment:** On 30 October 2026, the calculator and Nginx services were running and reported healthy. Port 8082 was unavailable, so the existing proxy was accessible at `http://localhost:8083`. The health endpoint returned `{"status":"UP"}`; the add endpoint returned a result of 5 for 2 + 3.

Architecture



The pipeline and deployment are configured in the project files. A successful Jenkins execution and external service configuration must be demonstrated separately; this report does not claim a completed Jenkins run.

IMPLEMENTATION

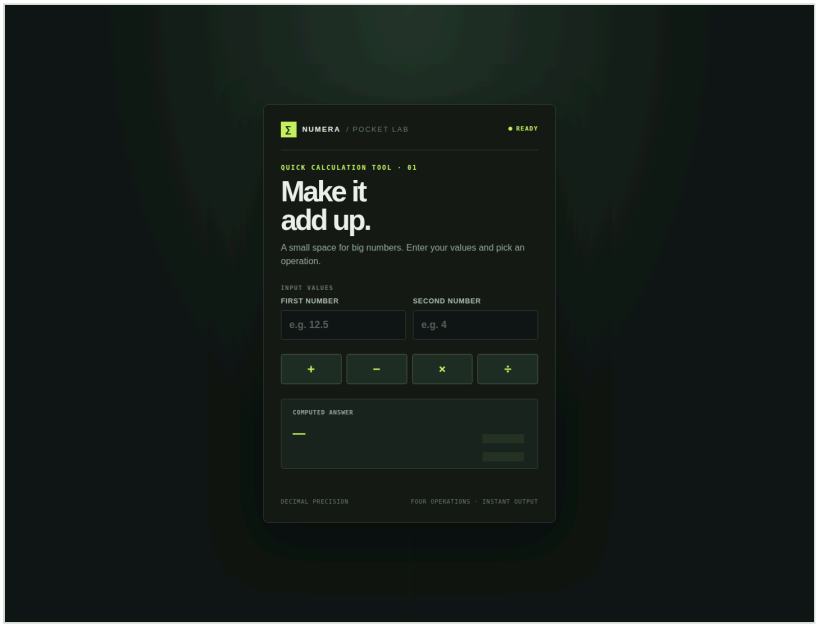
Pipeline and verification

Pipeline stages

Stage	What the project configures
Checkout	Checks out the revision supplied by the Jenkins multibranch job.
Build	Runs Maven clean package with tests skipped in this stage.
Test	Runs Maven tests and publishes Surefire JUnit XML reports.
Package	Packages the executable JAR and archives it in Jenkins.
Docker Build	Builds a branch/build-number tagged calculator image and the Nginx proxy image.
Push	On the main branch, authenticates with Jenkins-managed credentials and pushes the versioned and latest Docker Hub tags.
Deploy	On main, deploys through Compose and waits for service health checks.
Notifications	Jenkins post actions send success/failure email through the configured Email Extension settings.

Verification evidence

Compose health and API responses were captured from the running local stack. The UI screenshot below shows the calculator page served through the Nginx proxy.



Calculator interface served from the local Compose deployment at port 8083.

See `docs/evidence/local-verification.txt` for the recorded health response, sample API response, and Compose service status. The service test file covers all four arithmetic operations and divide-by-zero; the MVC test covers API responses and the frontend. Its page-heading assertion was updated to match the current UI. Tests were not rerun while preparing this report.

REVIEW AND HANDOFF

Challenges and remaining evidence

Challenges addressed

The frontend was redesigned after the original integration test had been written. That left an obsolete assertion expecting the old “Calculator” heading. The assertion has now been aligned with the current “Make it add up.” heading. The local Compose proxy also could not use its default host port 8082, so this verification used port 8083.

Items that still require account or live-run evidence

Item	Current evidence/status
GitHub repository and collaboration history	This local checkout currently has no commits or Git remote. Publish to the student’s repository and add progressive commits, a feature branch, and a merged pull request.
Jenkins run and trigger	Jenkinsfile and Jenkins Compose configuration are present. Configure the Jenkins job and GitHub webhook (or Poll SCM), then capture a successful run showing the stages.
Docker Hub image	The README names `certifiedUnc/spe-calculator` and the pipeline has push steps. Confirm the image is publicly available after an authenticated push.
Email proof	The Jenkinsfile has success/failure notification steps. Configure SMTP and Jenkins credentials, run the job, and capture the received notification.
Live demo	Demonstrate a GitHub push triggering an automatic build and deployment during the course review.

Conclusion

The application, container build, two-service Compose deployment, and declarative Jenkins pipeline configuration are in place. The local stack has been observed healthy and the API responds as expected. The final delivery still depends on publishing the repository and image, configuring Jenkins integrations, running the pipeline, and collecting the required account-level evidence.

> **Before submission:** run `mvn clean test`, confirm all tests pass, execute the Jenkins pipeline against GitHub, verify Docker Hub and email delivery, and save the Jenkins/email/container/API evidence screenshots.